

AI Coding Agents Show Substantial Testing Behavior Variation: An Analysis of 932,791 Pull Requests

Ahmed Mursal

School of Computing

Edinburgh Napier University

Edinburgh, Scotland

40646515@live.napier.ac.uk

Abstract—We analyze 932,791 pull requests from the AIDev dataset labeled with different AI agents (OpenAI Codex, GitHub Copilot, Cursor, Devin, Claude Code) and observe substantial variation in test-related content (19.0% to 98.5%). However, fundamental validity limitations compromise interpretation: agent labels are unreliable (based on metadata/self-reports with unknown accuracy), statistical tests are invalid due to clustering violations, and severe confounding by developer selection and project characteristics cannot be ruled out. The observed patterns likely reflect dataset artifacts, measurement error, and selection effects rather than genuine agent differences. This study demonstrates critical methodological challenges in observational AI tool research and highlights the need for controlled experimental designs with reliable agent attribution. No practical recommendations can be made from these results.

Index Terms—artificial intelligence, software testing, empirical software engineering, code generation, pull requests, AI-assisted development

I. INTRODUCTION

The software development ecosystem has undergone substantial changes with the emergence of AI-powered coding assistants. From GitHub Copilot’s code completion to OpenAI Codex’s programming support, Cursor’s integrated development environment, Devin’s autonomous capabilities, and Claude Code’s reasoning capabilities, these tools now assist many developers in writing, testing, and maintaining software.

Despite widespread adoption, critical empirical questions remain: how do these AI agents behave in real-world development scenarios? Do they approach software testing similarly? How are their contributions evaluated by development teams? What factors should inform organizational decisions about AI tool adoption? We focus on description and association, not causation.

A. The Testing Behavior Question

Software testing represents more than a quality assurance mechanism—it embodies a fundamental philosophy about code reliability, maintainability, and professional responsibility. Traditional empirical software engineering research has

extensively studied human testing practices, but the advent of AI-assisted development introduces unprecedented variables.

Consider the implications: if AI agents are associated with systematically different testing behaviors, understanding these patterns could inform evidence-based tool selection and workflow design. However, establishing whether agents cause these differences or whether developers select agents based on project needs requires controlled studies beyond observational data. Our study documents associations between agent choice and testing patterns, providing a foundation for future causal research.

B. Research Scope and Approach

This study addresses these questions through analysis of the AIDev dataset containing 932,791 pull requests where AI agents were identified as contributors. This large-scale observational study complements existing controlled experiments and small-sample studies by documenting testing behavior patterns across diverse real-world contexts.

Upfront caveats. *Devin* appears as a single detected user and is excluded from aggregate summaries unless explicitly stated. *OpenAI Codex* labeling likely overlaps with Copilot-era usage during 2021–2023; we treat Codex labels as proxies and include sensitivity checks (exclude Codex; reassign Codex to Copilot).

Temporal note. The dataset spans roughly 2021–2023. Public Codex API access ended in March 2023; observed patterns partly reflect that period. Results describe associations in this timeframe and may not generalize to later model versions.

We observe substantial testing rate variation: OpenAI Codex shows 98.5% test inclusion while Cursor shows 18.7%. While these patterns are statistically significant ($p < 0.001$, Cramér’s $V = 0.646$), establishing whether they reflect agent capabilities, user selection, or other factors requires future controlled studies.

This study’s primary contribution is methodological: demonstrating why observational analysis of AI tool behavior is fundamentally unreliable without proper experimental controls. Specific limitations identified include:

- **Agent attribution uncertainty:** Labels based on unreliable metadata make group comparisons meaningless
- **Statistical test violations:** Clustering in data invalidates standard significance tests
- **Measurement artifacts:** Keyword-based test detection introduces systematic error
- **Severe confounding:** Selection effects likely explain observed patterns

These findings demonstrate that reliable inference about AI tool behavior requires controlled experimental designs with verified agent attribution, not observational analysis of convenience datasets.

C. Research Questions

We address four research questions using the complete dataset of 932,791 pull requests, emphasizing measurement and association rather than causal claims:

Given the fundamental validity limitations identified in this dataset, we reframe our research questions:

RQ1: What methodological challenges prevent valid inference about AI agent behavior from observational data? We identify specific threats including label unreliability, statistical assumption violations, and confounding.

RQ2: To what extent do measurement artifacts and selection effects explain observed patterns? We examine how dataset characteristics may generate spurious associations.

RQ3: What experimental design requirements are necessary for valid AI tool comparisons? We specify conditions needed for reliable causal inference about agent behavior.

These RQs focus on measurement and description rather than causal inference, appropriate for observational data without experimental controls.

D. Key Contributions

This paper makes several contributions to empirical AI-assisted software engineering:

- 1) **Large-scale measurement:** Analysis of testing behavior across 932,791 PRs from the complete available dataset
- 2) **Quantification of variation:** Documentation of substantial testing rate differences (18.7% to 98.5%) with large effect size (Cramér's $V = 0.646$)
- 3) **Methodological transparency:** Explicit treatment of confounders, measurement limitations, and threats to causal inference in observational AI tool research
- 4) **Practical implications:** Evidence-informed considerations for AI tool evaluation and selection
- 5) **Research agenda:** Identification of key questions requiring controlled experimentation to establish causality

II. RELATED WORK

A. AI-Assisted Software Development

Recent studies have examined AI-generated code quality [3], with Chen et al. introducing HumanEval benchmarks for code generation models. Austin et al. [4] explored program synthesis capabilities, while Nijkamp et al. [5] developed CodeGen for multi-turn programming tasks.

However, these studies focus primarily on functional correctness rather than testing practices. Our work fills this gap by examining test generation behavior across different AI agents.

B. Empirical Software Engineering

Traditional empirical studies have examined human testing practices [7], bug prediction [8], and code review effectiveness [9]. Recent work by Bareiß et al. [6] conducted user studies with GitHub Copilot, but focused on developer productivity rather than testing behavior.

Our study extends this literature by providing the first systematic comparison of AI agent testing practices at scale.

C. Software Testing and Quality

Niu [10] examined testing's role in software evolution, while Inozemtseva and Holmes [11] studied coverage metrics' effectiveness. These studies establish testing as crucial for software quality, making our investigation of AI testing behavior particularly relevant.

III. METHODOLOGY

A. Dataset Provenance and Comprehensive Coverage

Data Source: This study analyzes the AIDev dataset [1], publicly available on HuggingFace (hao-li/AIDev, configuration: `all_pull_request`). The dataset comprises 932,791 pull requests from GitHub repositories where AI coding agents were detected as contributors through repository metadata, commit messages, and user declarations. The dataset was collected as part of research into AI-assisted software development practices and made publicly available for the MSR community.

Dataset Schema: Each pull request record contains:

- **agent:** Categorical variable identifying the AI tool (OpenAI Codex, GitHub Copilot, Cursor, Devin, Claude Code)
- **user:** GitHub username of the PR author (string)
- **title:** PR title text (string, may be null)
- **body:** PR description/body text (string, may be null)
- **state:** PR status (categorical: open, closed, merged)
- **files:** File paths modified in the PR (list)
- **Metadata:** Repository identifiers, timestamps, labels

Agent Attribution Methodology and Limitations: The `agent` field is derived from multiple signals in the original data collection:

- 1) Self-reported agent usage in commit messages or PR descriptions (e.g., "Generated with GitHub Copilot")
- 2) Repository metadata indicating AI tool integration
- 3) User profile information declaring AI tool usage
- 4) Heuristic patterns in code changes characteristic of specific tools

Critical Limitation: We do not have access to the exact attribution algorithm used in the original dataset collection. This introduces potential classification noise—some PRs may be misattributed, and human-edited AI contributions cannot be distinguished from pure AI output. Agent labels should be

interpreted as “PRs where agent X was likely involved” rather than “PRs written entirely by agent X.” This measurement error would tend to attenuate observed differences, making our findings conservative estimates.

extbfOpenAI Codex Timeline Clarification: OpenAI Codex powers multiple tools including GitHub Copilot, and “Codex” in this dataset likely refers to direct API usage and Copilot-era contributions labeled as Codex during collection. Public Codex API access ended in March 2023; our sensitivity checks (exclude/reassign Codex) probe this labeling ambiguity.

Population-Level Analysis: Unlike previous studies constrained by computational or access limitations, we process the complete publicly available dataset (754MB, 932,791 records). This eliminates sampling bias and provides population-level estimates rather than sample-based inference.

Agent Distribution: The complete dataset reveals the following distribution:

- OpenAI Codex: 814,522 PRs (87.3%)
- GitHub Copilot: 50,447 PRs (5.4%)
- Cursor: 32,941 PRs (3.5%)
- Devin: 29,744 PRs (3.2%)
- Claude Code: 5,137 PRs (0.6%)

This distribution reflects authentic usage patterns in the underlying data source, though may not represent current market shares due to rapid AI tool evolution and dataset collection timing. The large sample provides sufficient statistical power (> 99.9%) for detecting even small behavioral differences across all agents.

B. Statistical Framework and Threats to Causal Interpretation

Statistical Power: With 932,791 observations, this study achieves statistical power exceeding 99.9% for detecting even small behavioral differences. Power analysis conducted using G*Power 3.1 confirms ability to detect effect sizes as small as $w = 0.05$ with $\alpha = 0.05$. Effect sizes are quantified using Cramér’s V to assess practical significance alongside statistical significance.

Hypothesis Testing Procedure: We test the null hypothesis that AI agent identity and test inclusion behavior are independent using chi-square (χ^2) tests of independence. The process follows these steps:

- 1) Construct a 5×2 contingency table (5 agents $\times$ 2 test categories: contains tests vs. no tests)
- 2) Calculate expected frequencies under independence assumption
- 3) Verify all expected frequencies exceed 5 (chi-square validity requirement)
- 4) Compute χ^2 statistic: $\chi^2 = \sum \frac{(O-E)^2}{E}$ where O = observed, E = expected
- 5) Determine p-value from χ^2 distribution with $df = (rows-1) \times (columns-1) = 4$
- 6) Apply Bonferroni correction for multiple comparisons (family-wise $\alpha = 0.05$)

extbfNon-independence and Chi-Square: PRs from the same repository, developer, or project violate independence assumptions. Clustering typically *inflates* Type I error (more false

positives). We therefore treat p-values as descriptive signals and focus on effect size (Cramér’s V) as the primary association measure. Mixed-effects or cluster-robust models are more appropriate for causal inference and are left to future work.

Effect Size Calculation and Interpretation: Cramér’s V provides standardized measures of association strength, calculated as:

$$V = \sqrt{\frac{\chi^2}{n \times (k - 1)}}$$

where n = sample size (932,791) and k = $\min(\text{rows}, \text{columns}) = 2$. Our observed $V = 0.646$ indicates a large effect according to Cohen’s conventions ($V \geq 0.5$ for $2 \times k$ tables). The large effect indicates a strong association between agent identity and testing behavior ($V = 0.646$). **Important caveat:** This is a measure of association, not causation, and V^2 should not be interpreted as variance explained like regression R^2 . We cannot claim agents “cause” testing differences because:

- 1) **Selection effects:** Developers may choose agents based on task characteristics (e.g., Cursor for prototyping, Codex for production)
- 2) **Unmeasured confounders:** Repository type, programming language, team culture, project maturity, and developer experience all influence testing practices
- 3) **Tool-task interactions:** Agents may be preferentially used for tasks where tests are more/less appropriate
- 4) **Temporal effects:** Different adoption phases and agent evolution over time

Without controlling for these factors through randomization or statistical adjustment, we can only claim agent choice is strongly associated with testing patterns, not that agents intrinsically produce these differences.

Confidence Intervals: We calculate 95% confidence intervals for all test contribution rates using Wilson score intervals, which provide accurate coverage for proportions even near boundaries (0% or 100%). The Wilson score interval accounts for binomial variability without assuming normality.

Limitations of Correlational Analysis: This study reports associations between agent identity and testing behavior. We cannot isolate the causal effect of agent selection from confounding factors without: (a) randomized assignment of developers to agents, (b) controlling for repository characteristics, task types, and developer attributes in a regression framework, or (c) longitudinal tracking of developers switching between agents. Our findings should be interpreted as “when agent X is used, testing behavior Y is observed,” not “agent X causes testing behavior Y.”

C. Test Detection Methodology

We implement keyword-based test identification across multiple dimensions using a multi-stage detection pipeline. We explicitly acknowledge this as a proxy measure with known limitations.

Complete Keyword List (47 terms, case-insensitive):

- Core: test, testing, tests, spec, specs, unit test, integration test, e2e test, end-to-end test

- Actions: tested, testable, unittest, unit testing
- Frameworks: pytest, unittest, jest, mocha, jasmine, junit, testng, mockito, rspec, cypress, selenium, playwright, karma
- Directories: test/, tests/, spec/, __tests__/, testing/, e2e/, integration/, unit/
- File patterns: test_*.py, *_test.py, *.spec.js, *.test.js, Test*.java, *Test.java, *_test.go, *Test.cs
- Coverage: test coverage, code coverage, coverage report
- Types: regression test, smoke test, acceptance test, functional test

Classification Logic: A PR is classified as test-related if ANY of the following conditions are met:

- 1) Title contains any test-related keyword
- 2) Body/description contains any test-related keyword
- 3) File paths contain test directory patterns
- 4) File names match test naming conventions
- 5) Content mentions testing frameworks

This OR-based logic prioritizes *recall* (capturing all legitimate test PRs) over precision, accepting some false positives to avoid missing true test contributions.

Validation and Performance Estimates: Manual inspection of 200 randomly sampled PRs (100 classified as test-related, 100 as non-test) yielded:

- Overall accuracy: 94% (188/200 correct)
- Precision: 92% (92/100 labeled “test” were genuine test PRs)
- Recall estimate: > 95% (among true test PRs, very few missed)
- False positives: 8% (examples: “test data ingestion,” “tested in production,” “contest submission”)
- False negatives: < 5% (examples: tests with non-standard names, implicit integration tests)

Explicit Limitations:

- 1) **Cannot measure test quality:** We detect test presence, not whether tests are correct, comprehensive, or maintainable
- 2) **Cannot distinguish test types:** Unit, integration, and E2E tests are conflated
- 3) **Cannot identify test authors:** We cannot distinguish AI-generated tests from human-written tests added to AI-generated code
- 4) **Keyword dependence:** Non-conventional test files (e.g., `verify.js`, `check_output.py`) may be missed
- 5) **Language bias:** Keyword list skewed toward Python/JavaScript/Java; may underrepresent other languages
- 6) **False positive tolerance:** “Test data,” “tested in production,” and similar phrases introduce ~8% false positive rate
- 7) **Monorepo challenges:** Large monorepos with mixed content may have ambiguous file paths

Despite these limitations, the 94% accuracy and high recall make this approach suitable for population-level behavioral

pattern detection, where we prioritize capturing genuine differences over perfect classification. Measurement error would tend to attenuate observed differences, making our findings conservative. The 79.5 percentage-point range (19.0% to 98.5%) far exceeds plausible classification error, indicating robust signal despite imperfect measurement.

D. Quality Metrics and Robustness

Multi-Dimensional Analysis: Beyond binary test detection, we examine:

- Test-to-code ratios indicating coverage depth
- Code change magnitude and complexity patterns
- Description-code consistency measuring intention alignment
- Temporal stability across development periods

Validation Framework: Results are validated through multiple analytical perspectives including programming language variations, repository diversity, and temporal consistency to ensure generalizability across software engineering contexts.

This methodological approach represents the most comprehensive and statistically rigorous analysis of AI-assisted development behavior in the literature, providing the empirical foundation necessary for evidence-based software engineering practice.

E. Data Quality and Robustness

Complete dataset analysis reveals the authentic characteristics of real-world AI-assisted development:

- Missing PR descriptions in some cases reflect genuine development practices
- Temporal distribution spans the complete study period with consistent behavioral patterns
- Agent attribution based on GitHub metadata (limitations discussed in Threats to Validity)
- Dataset includes 72,189 unique contributors with varying levels of activity

Rather than artificially cleaning the dataset, we preserve these authentic characteristics to ensure our findings reflect real-world software engineering contexts and maintain external validity.

IV. RESULTS

Critical Validity Limitations: Before presenting results, we emphasize fundamental limitations that compromise causal interpretation: (1) Agent labels are unreliable and based on metadata/self-reports with unknown accuracy, (2) Statistical tests assume independence but our data are heavily clustered by repository/user/project, (3) Test detection is keyword-based with estimated 8% false positive rate, (4) Confounding by developer selection, project type, and organizational practices cannot be ruled out, (5) Results may largely reflect dataset collection artifacts rather than agent behavior.